

test

≡ 구분	Javascript
≡ 과목	

① GMS를 통한 API 호출 시, 엔드포인트를 GMS API로 변경 필요

- GMS API 엔드포인트 : <https://gms.ssafy.io/gmsapi/>

※ 엔드포인트 外 파라미터와 Body는 일반적인 AI API와 동일함

② 모델별 Authorization 위치에 Bearer Token을 GMS Key로 입력

- OpenAI 모델은 Header로 Bearer Token을 사용, Gemini 모델은 Query String으로 Bearer Token을 사용

REST API 방식 : OpenAI

```
# OpenAI의 gpt-4.1 모델을 호출할 경우 예시
curl "https://gms.ssafy.io/gmsapi/api.openai.com/v1/responses" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $GMS_KEY" \
-d '{
  "model": "gpt-4.1",
  "input": "Write a one-sentence bedtime story about a unicorn."
}'
```

REST API 방식 : Gemini

```
# Gemini의 gemini-2.0-flash 모델을 호출할 경우 예시
curl
"https://gms.ssafy.io/gmsapi/generativelanguage.googleapis.com
/v1beta/models/gemini-2.0-
flash:generateContent?key=$GMS_KEY" \
-H "Content-Type: application/json" \
-X POST \
-d '{
  ...
}'
```

1. OpenAI (gpt-4.1 모델 호출) 비교

OpenAI 방식은 열쇠(API Key)를 헤더(`-H "Authorization: ..."`) 안에 숨겨서 보낸다.

GMS 사용 안 할 때 (공식 OpenAI 호출) (평소 하던방식)

```
curl "https://api.openai.com/v1/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer sk-proj-0OpenAI공식키값" \
-d '{
  "model": "gpt-4-turbo",
  "messages": [{"role": "user", "content": "오늘점심뭐먹지"}]
}'
```

GMS 사용할 때 (GMS 사용 방식)

```
curl "https://gms.ssafy.io/gmsapi/api.openai.com/v1/responses" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $GMS_KEY" \
-d '{
  "model": "gpt-4.1",
  ...
}'
```

```
"input": "오늘점심 뭐먹지"
}'
```

달라진 점

1. **주소(URL):** `api.openai.com` 에서 `gms.ssafy.io/gmsapi/...` 로 목적지가 바뀌었다.
2. **인증 키:** 공식 키(`sk-proj-...`) 대신 SSAFY에서 준 `$GMS_KEY` 를 넣는다.
(넣는 위치는 헤더 `H` 로 동일)
3. **Body 데이터 규격:** GMS 내부 규격에 맞추어 `messages` 대신 `input` 형태로 바디 구성이 조금 바뀜

2. Gemini (gemini-2.0-flash 모델 호출) 비교

Gemini 방식은 열쇠(API Key)를 주소창 맨 뒤(`?key=...`)에 매달아서 보낸다.

GMS 사용 안 할 때 (공식 Gemini 호출) (평소 하던방식)

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key=AIzaSyGoogle공식키값" \
-H "Content-Type: application/json" \
-X POST \
-d '{
  "contents": [{
    "parts":[{"text": "오늘 점심 메뉴 추천해줘"}]
  }]
}'
```

GMS 사용할 때 (GMS 사용 방식)

```
curl "https://gms.ssafy.io/gmsapi/generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key=$GMS_KEY" \
-H "Content-Type: application/json" \
-X POST \
-d '{
  "contents": [{
    "parts":[{"text": "오늘 점심 메뉴 추천해줘"}]
  }]
}'
```

달라진 점 체크

1. **주소(URL):** 원래 주소 앞에 `https://gms.ssafy.io/gmsapi/` 가 배대지 주소처럼 추가로 붙었다.
2. **인증 키:** 주소창 맨 끝 쿼리 스트링(`?key=`) 자리에 공식 구글 키 대신 SSAFY에서 발급받은 `$GMS_KEY` 를 그대로 꽂아 넣는다.

③ SDK 방식이나 Langchain 방식으로도 GMS Key 활용이 가능

- 라이브러리를 우선 설치하고, 발급 받은 GMS의 Key 값과 모델별 Base_url을 입력하여 사용

SDK 방식 : OpenAI

```
# SDK 방식으로 OpenAI API를 호출하는 예시
# OpenAI 라이브러리 설치
!pip install openai

from openai import AsyncOpenAI
import os
import getpass
import asyncio

# GMS KEY를 입력
if not os.environ.get("OPENAI_API_KEY"):
    os.environ["OPENAI_API_KEY"] = getpass.getpass("GMS KEY를 입력하세요:")

# OpenAI의 BASE_URL을 GMS로 설정
client = AsyncOpenAI(base_url="https://gms.ssafy.io/gmsapi/api.openai.com/v1")

...
```

Langchain 방식 : OpenAI

```
# Langchain 방식으로 OpenAI API를 호출하는 예시
# Langchain 라이브러리 설치
!pip install langchain

import getpass
import os
from langchain.chat_models import init_chat_model

# GMS KEY를 입력하는 단계
if not os.environ.get("OPENAI_API_KEY"):
    os.environ["OPENAI_API_KEY"] = getpass.getpass("GMS KEY를 입력하세요:")

# langchain의 BASE_URL을 GMS로 설정
os.environ["OPENAI_API_BASE"] =
    "https://gms.ssafy.io/gmsapi/api.openai.com/v1"

...
```

1. SDK 방식 (OpenAI 라이브러리 사용) 비교

기본적으로 `AsyncOpenAI` 라이브러리는 미국 OpenAI 본사 서버 주소가 내장되어 있다.

GMS를 쓸 때는 이 목적지(`base_url`)를 강제로 바꿔주는 것이 핵심이다.

GMS 사용 안 할 때 (공식 OpenAI SDK)

```
import os
import asyncio
from openai import AsyncOpenAI

# 공식 OpenAI API 키를 환경변수에 등록
os.environ["OPENAI_API_KEY"] = "sk-proj-0OpenAI공식키값"

# 별도의 주소 설정 없이 클라이언트를 생성 (자동으로 공식 서버로 연결됨)
client = AsyncOpenAI()

async def main():
    response = await client.chat.completions.create(
        model="gpt-4-turbo", # 공식 OpenAI 모델명
        messages=[{"role": "user", "content": "오늘 점심 메뉴 추천해줘"}]
    )
    print(response.choices[0].message.content)

asyncio.run(main())
```

GMS 사용할 때

```
import os
import asyncio
from openai import AsyncOpenAI

# 1. SSIFY에서 발급받은 GMS KEY를 환경변수에 등록
os.environ["OPENAI_API_KEY"] = "발급받은_GMS_키값"

# 2. 핵심: 내비게이션 목적지(base_url)를 GMS 우회 주소로 변경!
```

```

client = AsyncOpenAI(
    base_url="https://gms.ssafy.io/gmsapi/api.openai.com/v1"
)

async def main():
    # 3. 호출 (GMS 주소와 키가 세팅되어 있어 알아서 우회하여 전송됨)
    response = await client.chat.completions.create(
        model="gpt-4.1", # SSAFY GMS 전용 모델명
        messages=[{"role": "user", "content": "오늘 점심 메뉴 추천해줘"}]
    )
    print(response.choices[0].message.content)

asyncio.run(main())

```

2. Langchain 방식 비교

랭체인은 개별 코드에 주소를 적지 않고,

시스템 환경변수를(`OPENAI_API_BASE`) 바꿔준다.

GMS 사용 안 할 때 (공식 Langchain)

```

import os
from langchain.chat_models import init_chat_model

# 공식 OpenAI API 키 등록

os.environ["OPENAI_API_KEY"] = "sk-proj-OpenAI공식키값"

# 모델 초기화 (기본 내장된 공식 주소로 찾아감)

gpt_model = init_chat_model("gpt-4-turbo", model_provider="openai")

response = gpt_model.invoke("오늘 점심 메뉴 추천해줘")
print(response.content)

```

GMS 사용할 때

```

import os
import getpass
from langchain.chat_models import init_chat_model

# 1. GMS KEY를 환경변수에 등록 (키가 없으면 콘솔 창에서 입력받는 로직 적용)

if not os.environ.get("OPENAI_API_KEY"):
    os.environ["OPENAI_API_KEY"] = getpass.getpass("GMS KEY를 입력하세요: ")

# 2. 핵심: 랭체인이 읽고 찾아갈 기본 주소 표지판(BASE)을 GMS 주소로 변경!

os.environ["OPENAI_API_BASE"] = "https://gms.ssafy.io/gmsapi/api.openai.com/v1"

# 3. 모델 초기화 (위에서 바꾼 환경변수 표지판을 보고 자동으로 GMS로 우회함)

gpt_model = init_chat_model("gpt-4.1", model_provider="openai")

response = gpt_model.invoke("오늘 점심 메뉴 추천해줘")
print(response.content)

```

달라진 점 체크

- **SDK 방식 차이점:** `base_url="..."` 설정을 추가하여 기본 내장 주소를 GMS 주소로 들어주는 것이다.
- **Langchain 방식 차이점:** `os.environ["OPENAI_API_BASE"]` 환경변수를 한 줄 더 추가해서 GMS로 덮어씌우는 것이다.

이 설정을 통해, 복잡한 주소나 헤더 구조를 매번 타이핑하지 않고도 **기존 오픈소스 코드를 고스란히 유지하면서 GMS 서비스를 편리하게 이용할 수 있게 되는 것이다.**